

New South Wales Oil Price Analysis

1. Data retrieval

The entire process uses requests for HTTP requests, lxml and XPath to parse HTML, BeautifulSoup to extract page elements further, and Pandas to read and merge data. Thus, a complete and efficient automated data crawling and integration process is realized.

First used the requests. The get () method from the NSW data set home page (<https://data.nsw.gov.au/data/dataset/fuel-check>) for HTML content. The HTML string is converted into a parse tree via etree, and the element containing the data resource is located using the XPath expression //ul[@class='resource-list']/li. Each element is then iterated over, and in each loop the <a> tag's href attribute is extracted via XPath to get the link to the data detail page, concatenated with the main domain name "https://data.nsw.gov.au" to form the full URL. In the data details page, look for the <a> tag with the "resource-url-analytics" class and extract a link to the real data file from it. Considering that the file format is xlsx or csv, the data is read using pd.read_excel() first; If this fails, use pd.read_csv() to read. All the read data is merged and the index is reset. Finally, the integrated data is exported as the combine_total.csv file.

2. Data Augmentation

In order to support the geographic visualization required in the second phase, we need additional information in addition to the original dataset. We obtained two external datasets: an Australian postcode and location dataset (postcode.txt), which includes postcodes, suburbs, cities, and geographic coordinates of suburbs, and gas station geolocation dataset (station_location.txt), which contains the latitude and longitude of each station extracted from the original dataset. This is crucial for implementing location-based queries.

The original dataset contains postcodes and suburbs, but because we need to do city-level statistics in the next stage, we need to know which suburbs belong to which cities, so finding their mapping is necessary. We downloaded a dataset containing Australian postcodes, suburbs, their latitude and longitude, and the city they belong to from geonames.org. This helped us integrate the suburb data into city-level data and provided the geographic location data of the suburbs required in the geographic visualization. We filtered out the postcodes that appeared in the original dataset because the downloaded dataset contains a large number of locations in other states, which is meaningless for our analysis of NSW data. It is worth noting that because the original dataset retains the ACT data, we did not delete the ACT data. In the end, we got a mapping of the postcodes and cities and their geographical locations only in the original dataset.

Since the gas stations in the original dataset only have addresses but no geographic locations, it will not be possible to display the locations of the gas stations on the map in the second phase, so we need to collect the coordinates of each gas station. We tried many APIs, but most of them were not ideal (the API did not include gas stations, or the longitude and latitude were accurate only to the postcode level), so we finally used the Google Maps Geocoding API to resolve the gas station addresses into precise longitude and latitude values. Before querying the API, we deduplicated the gas station addresses to reduce the number of requests. In the end, we got the exact addresses of all gas stations, except for two gas stations that were not mapped to coordinates by Google Maps. So we

manually entered the names of the gas stations and found their coordinates. Finally, we got the complete latitude and longitude data of the gas stations.

In summary, the reason for collecting these two additional datasets is to support our next phase of more accurate visualization on the map and enhance the possibility of more ways to analyze fuel prices.

3.Data Integration

3.1 Postcode map cities:

In order to analyze fuel prices for each city in New South Wales, the data in the postcode columns of the 15-month dataset from 2024 to March 2025 were first mapped to the corresponding city names and geographic information (latitude and longitude). By extracting all unique postcodes that appeared in the dataset and reading an auxiliary file "postcode.txt" containing the postcode, city name, and geographic coordinates, this auxiliary file is a table of unique mappings from postcodes to cities and latitude/longitude. This process provides important data to rely on for subsequent visualization and analysis of fuel prices in cities.

3.2 Build StationBrand Table and FuelType Table:

To extract all gas station brand and fuel type information from the original dataset, create the brand dimension table StationBrand and the fuel type dimension table FuelType. The brand dimension table: extracts all unique brand names that appear in the dataset and generates a primary key as a unique identifier (brand_id) for each brand, with the brand field renamed to brand_name. The final result is the creation of a cleaned up data table, data_brand, which contains only the brand_id and brand_name. Fuel Type dimension table: extract all unique fuel codes that appear in the dataset and generate a primary key as a unique identifier (fuel_id) for each fuel type, with the FuelCode field renamed to fuel_code. create a cleaned data table containing only brand_id and brand_name. the final result is to create a cleaned data table (data_fuel) that contains only fuel_id and fuel_code.

3.3 Build ServiceStation Table:

First an auxiliary data file station_location.txt is introduced to contain the Address, Latitude and longitude of each gas station. Then in order to construct the gas station dimension table ServiceStation, the basic information of each gas station is extracted from the original data, which include ServiceStationName, Address, Suburb, Postcode, and Brand. Finally, the gas station data table data_station containing station_id, name, address, postcode, brand_id is generated. station with the address field and the geographic location data file station_location. txt with the corresponding latitude and longitude. txt with corresponding latitude and longitude to form the final ServiceStation dimension table.

3.4 Build Location Table:

In order to analyze the geographic information in the city, a geographic location dimension table Location is constructed based on postcode, and contains the information of postcode, suburb, city, latitude and longitude, etc. Firstly, the postcode and suburb fields are extracted from the main dataset, and then the basic location table is constructed after de-weighting. The complete city name and geographic coordinates are matched by left joining the geographic location dimension table Location

with the data_city table through the postcode field. The resulting Location table will be used as the city geographic information dimension table.

3.5 Build FuelPriceFact Table:

The fuel price information from the original dataset is utilized to construct the final fact table FuelPriceFact, which is used to record each specific fuel price change information. The table contains record_id, station_id, fuel_id, price_updated_time, and price. First, we joined the raw fuel price data table with the gas station dimension table ServiceStation via station name, address, and zip code. Then, based on the oil type dimension table FuelType, a mapping of fuel_code to fuel_id is created. Finally, we filter the field information to be retained and form the structured and visualized FuelPriceFact table.

3.6 Fuel type processing:

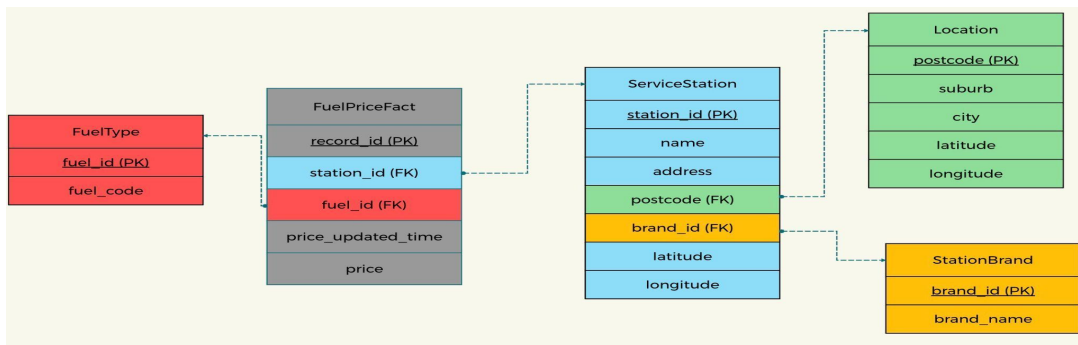
From the perspective of oil product types, the data includes 9 types of oil products, but there are serious missing problems. The record times of the two oil products, E85 and B20, were only 76 and 2 respectively, which almost could not support any meaningful analysis. In addition, the total number of records for the other two types of oil products is also less than 100,000, indicating limited data coverage. Therefore, in the end, we only retained the remaining five mainstream oil products (E10, P95, P98, PDL and U91) with more than 100,000 records as the objects for subsequent analysis. When further processing the data of these five types of oil products, we noticed that not all gas stations update the price of oil products every day. In order to solve this problem, we calculate the average price of the same oil product for all gas stations in the same city on the same day in terms of “city-date-oil product” as the representative price of the oil product in the city on the same day, so as to alleviate the problem of uneven data at the gas station level. However, even after completing the aggregation operation of daily averages, there are still a large number of missing records. We found that these missing values are not nulls in the traditional sense, but structural missingness due to the complete absence of records on certain dates.

3.7 Average fuel price processing:

In order to minimize the impact of such structural deficiencies on the subsequent analysis, we further performed month-by-month aggregation by calculating the monthly average price of each oil product in each city. In this way, we were able to show more stable price volatility trends at the city level. On this basis, we screened the data for high temporal completeness, eliminating cities with discontinuous monthly data and more than one missing month, and retaining only those cities that covered the full 15-month period. This process ensures continuity and reliability for subsequent trend analysis and visualization.

4. Database Design

To effectively manage and analyze the fuel price data of New South Wales, we have designed the Snowflake model database. Among them, FuelPriceFact is a fact table, and FuelType, ServiceStation, StationBrand, and Location are four-dimensional tables. DayResult and MonthResult are derived tables to prepare for the subsequent drawing. Each table has normalization, scalability and analytical flexibility.



- The central table FuelPriceFact captures raw fuel price observations over time. It contains fields such as station_id, fuel_id, price_updated_time, and price. It serves as the fact table for tracking price changes and supports temporal analysis.
- FuelType normalizes fuel identifiers (e.g., U91, P95) via fuel_id, ensuring consistency and avoiding duplication across records.
- ServiceStation provides detailed information on individual fuel stations, including their name, address, postcode, and associated brand_id.
- StationBrand maps each brand_id to a brand_name such as BP, Shell, or Costco, supporting brand-level aggregations.
- Location maps postcode values to geographical attributes like suburb, city, and geographic coordinates (latitude, longitude), enabling spatial queries and geovisualization.
- Two derived summary tables:
 - DayResult: stores daily average fuel prices grouped by city, fuelcode, and date_only.
 - MonthResult: stores monthly average fuel prices grouped by city, fuelcode, and date_month.

These tables are interconnected through primary and foreign keys, ensuring referential integrity (e.g., station_id, fuel_id, brand_id, and postcode). This database design reflects real-world relationships and supports efficient querying, making it suitable for time-series tracking, brand comparisons, and spatial fuel price analysis.

5. Contribution Report

Fuhai Liang: Responsible for data collection, integration and warehousing, and completing the visualization of monthly price changes of different oil products in various cities.

Nan Chen: Responsible for constructing the daily oil price interactive line chart and handling the data integration required for the report.

Lang Xu: Responsible for constructing the interactive visual chart of weekly oil prices and participating in the design of the report data structure.

Qiuwen Li: Responsible for visualizing the interactive box charts of oil prices in different cities and conducting data analysis to provide support for report writing.

Bowen Bai: Data enhancement, obtaining the longitude and latitude of a city through its postal code. Obtain the longitude and latitude of the gas station through the address